

# Compositional Semantics for Parametric Linear Programming, a Categorical Approach

Bruno Llacer Trotti      João Paixão      Iago Leal

February 2026

## Abstract

We give a compositional, categorical account of right-hand side parametric linear programming. Building on the graphical calculi for linear, affine and polyhedral algebra, we introduce GLP, a symmetric monoidal theory that extends Graphical Polyhedral Algebra with a single cost generator encoding a linear objective. We show that its morphisms are exactly the parametric linear programs — equivalently, the polyhedral convex functions — and prove that GLP soundly and completely axiomatises this semantics. The development lifts linear programming from a numerical problem to an algebraic object living in the Extended-Lawvere quantale of costs, where the elimination of internal variables is exactly categorical composition.

## 1 Introduction

Mathematical and physical theories have grown increasingly difficult to express through purely symbolic notation, prompting engineers and scientists to develop quasi-formal diagrammatic languages for recording their designs and ideas, from electrical circuit diagrams [BF18] to Feynman diagrams and control-system block diagrams. More recently, this diagrammatic spirit has reached mathematics and computer science through the notion of string diagrams from Category Theory, giving rise to diagrammatic algebras that axiomatise reasoning within specific domains, such as the ZX-calculus for quantum computing [CD11]. One such development is the still-growing field of Graphical Algebras, and particularly its more expressive polyhedral variants [BDS21].

This work aims to lift such algebras to an even higher level of expressivity: turning the already complete theory of polyhedra into a more general language by extending it towards linear problems. We observe that many mathematical structures used in optimization can be understood within a single compositional framework [SS24; Han+24]. Instead of treating convex functions, polyhedra, and linear programs as distinct objects, we show that they can all be interpreted as morphisms (relations) in a suitable category whose composition is given by

elimination or minimization. In this way, optimization itself becomes an algebraic process and a universal setting across mathematics.

Over that goal, the language of string diagrams [JS91; Sel10; PZ23] help us to make this expressivity even more aparent. Acting as rigorous framework for monoidal categories, string diagrams provide an intuitive yet mathematically precise syntax in which the sequential and parallel composition of morphisms correspond directly to vertical and horizontal juxtaposition of diagrams [JS91; Lan98].

Building on this foundation, a rich family of *diagrammatic* or *graphical calculi* has emerged, offering complete axiomatisations of various algebraic structures purely through equational diagram rewriting [Sel10; BSZ17]. These calculi have proved remarkably versatile, finding applications across a wide range of fields: in *quantum computing* [CD11]; *electronics* [BF18]; *linear algebra* [BSZ17; Bon+19; Fre+25]; and in *optimisation and probability* [Ste+24; SS21; BDS21]. Throughout this work we explore how, step by step, one can enrich the already known theories for linear [BSZ17] algebra by adding one new generator at a time, along with a particular set of axioms, and derive even richer structures for convex problems [SS24].

A first instance of this mechanism is the Graphical Quadratic Algebra (**GQA**) [Ste+24], where the addition of a single “measurement” generator expands the expressivity of the previous theories by acting as a sort of measurement device along the diagram, lifting diagrams from the class of binary relations to the broader class of weighted relations. From the optimization perspective, this is what plays the role of the objective function in an optimization problem. We do not develop the quadratic branch here, but it serves as the conceptual template for the cost generator at the heart of our contribution.

The polyhedral branch is the one our work builds on directly. Extending the Symmetric Monoidal Theory of Graphical Affine Algebra (**GAA**) [Bon+19] with an inequality generator  $\text{---}\big(\geq\big)\text{---}$  yields Graphical Polyhedral Algebra (**GPA**) [BDS21]. Geometrically, this allows the algebra to go beyond affine spaces and start representing polyhedral figures as linear inequality constraints in optimization,

$$\{|x \in P|\} = \begin{cases} 0 & \text{if } Ax + b \geq 0 \\ +\infty & \text{otherwise.} \end{cases}$$

Inspired by these observations, we make our central contribution: the design of a graphical algebra for parametric linear programming through the introduction of a new generator  $\text{---}\square\text{---}$  and three axioms. In this setting, a linear program is no longer viewed only as a numerical problem to be solved, but as a morphism that can be composed with other optimization blocks. This gives a symbolic language in which the elimination of internal variables corresponds directly to categorical composition, allowing optimization problems to be manipulated diagrammatically before computation.

In particular, every piecewise affine convex function can be represented as the value function of a linear program [Roc70]. This means that **GLP** provides a syntax for a broad class of nonsmooth convex models arising in optimization,

control, economics, and machine learning [BV04] — including feedforward network architectures, electrical circuits, and control-feedback systems. In Section 3 we show that these two classes of objects are not only denotationally equivalent but instances of the same normal form. Thus, our diagrammatic syntax blurs the distinction between expressing a linear program as an optimization problem and as a piecewise affine convex function, deriving a canonical normal form.

More broadly, this work suggests that graphical methods can play for convex optimization the same role that graphical linear algebra plays for matrices: providing normal forms, symbolic rewriting rules, and completeness results within a rigorous categorical setting. This opens the possibility of using diagrammatic reasoning not only for proofs, but also for designing compositional optimization systems and formal symbolic calculi for convex programming.

**Overview** Section 2 introduces the algebraic and categorical setting on which the rest of the work is built: the initial notions of quantales and relations, the Extended-Lawvere quantale of costs, the language of symmetric monoidal theories, and the prior graphical theories **GLA**, **GAA** and **GPA** on which our contribution rests. Section 3 introduces, for the first time, the categorical semantics of parametric linear programs, establishing the equivalence between piecewise affine convex functions, polyhedral functions and parametric linear programs, and packaging them as the categories **LPRel** and **PolyFun**. Section 4 defines the symmetric monoidal theory **GLP**, its normal form, and the epigraph form. Section 5 proves soundness, expressivity and completeness, establishing that **GLP** axiomatises the semantics. This work is a step toward a complete diagrammatic calculus for convex optimization.

## 2 Background

In this section we present the background theory for formalizing diagrammatic algebras and the constructions needed to do so. We also give a brief introduction to previous works to provide the necessary notions and results that will be used later.

This section is outlined as follows: firstly, we present the background theory of quantales and how they provide a suitable theory and algebraic form to express relations in a general sense. Secondly, we offer a brief introduction to the categorical field of formal semantics and monoidal theories and how they are used to build the syntax of diagrammatic calculi. Lastly, we present the existing theories on which our work is built, namely Graphical Linear Algebra [PRS22; BSZ17], Graphical Affine Algebra [Bon+19] and Graphical Polyhedral Algebra [BDS21; BDZ21].

### 2.1 Preliminaries on relations

While much of mathematical theory is built on functional thinking, we argue that most of mathematics actually has a relational nature. Category theory is

the natural language for this purpose. It is precisely built on top of this notion, in which everything one needs to know about an object is determined by its morphisms.

To begin we define the most basic kind of relation:

**Definition 1** (Binary Relations). *Let  $X$  and  $Y$  be sets. A binary relation  $R$  from  $X$  to  $Y$  is a subset*

$$R \subseteq X \times Y.$$

*Equivalently, it can be identified with its characteristic function*

$$\chi_R : X \times Y \rightarrow \{0, +\infty\},$$

*defined by*

$$\chi_R(x, y) = \begin{cases} 0 & \text{if } (x, y) \in R, \\ +\infty & \text{otherwise.} \end{cases}$$

A binary relation specifies whether  $x$  is related to  $y$ . One can think of a relation in two different ways that will prove useful later: first, as a set of pairs, where every relation is just a pair  $(x, y) \in R$ , or as a map that has  $x, y$  as inputs and outputs  $0$  if they are related, and  $+\infty$  if they are not (the choice of  $\infty$  will be justified later; normal choices here are actually  $\{\text{true}, \text{false}\}$  or  $\{0, 1\}$ , the latter being isomorphic to our choice).

This is a general construct, as the sets  $X$  and  $Y$  can be anything. We can, for instance, ask for further structures on such sets and obtain different kinds of relations.

**Definition 2** (Linear/Affine Relations). *Let  $X$  and  $Y$  be vector spaces. A linear relation from  $X$  to  $Y$  is a linear subspace*

$$R \subseteq X \times Y.$$

*An affine relation is an affine subspace of  $X \times Y$ . Equivalently, a linear relation can be described as the graph of a possibly multivalued linear operator, i.e.,*

$$R(x) := \{y \in Y \mid (x, y) \in R\}.$$

Thus, a linear/affine relation is a binary relation where the set  $R$  is a subspace. We have so far restricted relations between  $x$  and  $y$  to a binary “related or not” notion. In fact, we might be interested in adding degrees to such a relation, so that we could say that  $x$  and  $y$  are “more” or “less” related. This leads us to the well-known notion of weighted relations [Law73].

**Definition 3** (Weighted Relations). *Let  $X$  and  $Y$  be any two sets and let  $\overline{\mathbb{R}}^+ := [0, +\infty]$ . A weighted relation from  $X$  to  $Y$  is a set*

$$R \subseteq X \times Y \times \overline{\mathbb{R}}^+$$

*such that for every  $(x, y) \in X \times Y$  there exists exactly one  $r \in \overline{\mathbb{R}}^+$  satisfying  $(x, y, r) \in R$ . Equivalently, we may define the characteristic function  $\chi_R : X \times Y \rightarrow \overline{\mathbb{R}}^+$  by  $\chi_R(x, y) = r$  if and only if  $(x, y, r) \in R$ .*

In fact, one could choose any other set instead of  $\overline{\mathbb{R}}^+$  to be the codomain of the characteristic function, as long as it satisfies some properties. The whole structure of a relation is further generalized by the notion of a quantale [Ros90].

**Definition 4** (Quantale). *A quantale is a structure  $(Q, \leq, \otimes, e)$  such that:*

- $(Q, \leq)$  is a complete lattice, i.e., every subset  $A \subseteq Q$  admits a join  $\bigvee A$  and a meet  $\bigwedge A$ ;
- $(Q, \otimes, e)$  is a monoid;
- Multiplication distributes over arbitrary join in each variable:

$$a \otimes \left( \bigvee_{i \in I} b_i \right) = \bigvee_{i \in I} (a \otimes b_i), \quad \left( \bigvee_{i \in I} a_i \right) \otimes b = \bigvee_{i \in I} (a_i \otimes b)$$

for all families  $\{a_i\}_{i \in I}, \{b_i\}_{i \in I} \subseteq Q$ .

If  $\otimes$  is commutative, the quantale is called commutative. If  $e = \top$  (the top element of the lattice), it is called integral.

Given a quantale  $Q$ , we can define the notion of  $Q$ -valued relation [Coe+18].

**Definition 5** (Generalized Relations). *A generalized relation (or  $Q$ -valued relation) from  $X$  to  $Y$ , where  $Q$  is a quantale, is a set  $R \subseteq X \times Y \times Q$  such that for every  $(x, y) \in X \times Y$  there exists exactly one  $q \in Q$  satisfying  $(x, y, q) \in R$ . Its characteristic function is  $\chi_R : X \times Y \rightarrow Q$  defined by  $\chi_R(x, y) = q$  if and only if  $(x, y, q) \in R$ .*

The notion of a  $Q$ -valued relation recovers familiar concepts for particular choices of quantale. For the (Boolean) truth quantale  $Q = \{0, 1\}$  with  $\otimes = \wedge$  and  $e = 1$ ,  $Q$ -valued relations coincide with classical binary relations. For the Lawvere quantale  $Q = \overline{\mathbb{R}}^+$  ordered by the reverse of the usual order, with  $\otimes = +$  and  $e = 0$ , a  $Q$ -valued relation is precisely a weighted relation assigning a (possibly infinite) cost to each pair. Hence generalized relations unify ordinary binary relations and weighted relations as special cases corresponding to particular quantales.

Within this framework, we propose an extension of the Lawvere Quantale and weighted relations called the Extended-Lawvere Quantale and the Cost/Reward Relations, respectively.

**Definition 6** (Optimization/Extended Lawvere Quantale). *Let  $\overline{\mathbb{R}} := \mathbb{R} \cup \{-\infty, +\infty\}$ . The optimization quantale is the structure  $(\overline{\mathbb{R}}, \geq, \otimes, e)$  defined as follows:*

- The order  $\geq$  is the usual order on the extended real line. With this order, arbitrary joins are given by the usual infima:  $\bigvee A = \inf A$ .
- The monoidal product is addition  $a \otimes b := a + b$ , where addition is extended in the usual way (with  $a + (+\infty) = +\infty$ ,  $a + (-\infty) = -\infty$  and  $(+\infty) + (-\infty) = +\infty$ ).

- The unit element is  $e := 0$ .

With these operations,  $(\overline{\mathbb{R}}, \geq, +, 0)$  is a commutative quantale. Joins are ordinary infima, and addition distributes over arbitrary joins.

**Definition 7** (Optimization Cost Relations). *Let  $X$  and  $Y$  be sets. An optimization relation from  $X$  to  $Y$  is an  $\overline{\mathbb{R}}$ -valued relation, i.e., a function  $R : X \times Y \rightarrow \overline{\mathbb{R}}$ . The value  $R(x, y)$  is interpreted as the objective value (cost) associated to the pair  $(x, y)$ , with  $+\infty$  and  $-\infty$  representing extremal infeasibility or unboundedness.*

This extends weighted relations to include negative values. The importance of it is that we want to be able to express optimization problems that might be infeasible ( $+\infty$ ) or unbounded ( $-\infty$ ), so we need to add both the top and bottom element. This is also the correct setting to discuss problems such as *flow networks* or *quantitative formal concepts*, where a notion of cost or reward can be assigned to transitions between elements.

All of this yields a suitable and natural categorical construction. Specifically, from any quantale  $Q$  one can not only define  $Q$ -relations, but also construct an entire category of them. This category is the natural setting for discussing the central concept of this framework: the composition of relations. The following proposition shows how to build such a category and establishes that it carries a very important monoidal structure; particularly, it is a compact closed symmetric monoidal category [Lan98].

**Proposition 1** (Category  $\mathbf{Rel}(Q)$ ). *For a quantale  $Q$ , the  $Q$ -relations form a category  $\mathbf{Rel}(Q)$  with composition and identities*

$$(S \circ R)(a, c) = \bigvee_b R(a, b) \otimes S(b, c), \quad 1_A(a, b) = \bigvee \{e \mid a = b\}.$$

*If  $Q$  is a commutative quantale, then  $\mathbf{Rel}(Q)$  carries a symmetric monoidal structure, with the categorical monoidal product  $\otimes_c$  on objects given by the Cartesian product of sets and the action on relations given for  $R : A \rightarrow C$  and  $S : B \rightarrow D$  by*

$$(R \otimes_c S)(a, b, c, d) = R(a, c) \otimes S(b, d).$$

*The singleton set is the monoidal unit, and the category is compact closed w.r.t. the monoidal structure.*

*Proof.* Associativity follows from the distributivity of  $\otimes$  over  $\bigvee$ : for  $R : A \rightarrow B$ ,  $S : B \rightarrow C$ ,  $T : C \rightarrow D$ ,

$$((T \circ S) \circ R)(a, d) = \bigvee_{b, c} R(a, b) \otimes S(b, c) \otimes T(c, d) = (T \circ (S \circ R))(a, d).$$

The identity  $1_A(a, b) = e$  if  $a = b$  and  $\perp$  otherwise satisfies  $R \circ 1_A = R$  since  $e \otimes R(a, b) = R(a, b)$  and all remaining terms vanish. When  $Q$  is commutative, the symmetric monoidal structure is given by Cartesian product on objects and  $(R \otimes_c S)((a, b), (c, d)) = R(a, c) \otimes S(b, d)$  on morphisms; functoriality follows from

commutativity and distributivity. Compact closure is witnessed by the unit and counit  $\eta(*, (a, a')) = \varepsilon((a, a'), *) = e$  if  $a = a'$  and  $\perp$  otherwise, for which the triangle identities hold by direct computation. We refer to [Ros90] for full details.  $\square$

Relations encode the notion of matrices and matrix multiplication. A  $Q$ -valued relation  $R : X \times Y \rightarrow Q$  is nothing but a  $Q$ -valued *matrix* indexed by  $X \times Y$ , and composition of  $Q$ -valued relations corresponds precisely to matrix multiplication in  $Q$ , where ordinary addition is replaced by the join  $\bigvee$  and ordinary multiplication by the monoidal product  $\otimes$ . Worth noticing, however, that in this case, matrices can indeed be infinite (countable), different from the usual finite case. The two axioms of a quantale — that  $\otimes$  distributes over arbitrary joins — are precisely what guarantees that this multiplication is associative and well-defined. This serves as motivation for what will later become the diagrammatic theory of linear algebra.

The resulting categories are symmetric monoidal, and it is precisely this monoidal backbone that will support the algebraic structure we aim to build. A key piece of that structure is a *Frobenius monoid* on each object: a monoid  $(\mu, \eta)$  and a comonoid  $(\delta, \varepsilon)$  on the same object, satisfying the Frobenius law

$$(\text{id} \otimes \mu) \circ (\delta \otimes \text{id}) = \delta \circ \mu = (\mu \otimes \text{id}) \circ (\text{id} \otimes \delta).$$

A symmetric monoidal category in which every object carries such a structure, compatibly with the tensor product, is called a *hypergraph category* [FS19]. Hypergraph categories are related to, but strictly more general than, compact closed categories [Sel10], and it is the more general notion that we adopt as our ambient framework.

**Remark 1.** *If  $Q$  is commutative, then  $\mathbf{Rel}(Q)$  is a hypergraph category [FS19] and admits a commutative Frobenius structure on every object. Diagrammatically, we denote the additive and co-additive structure as*

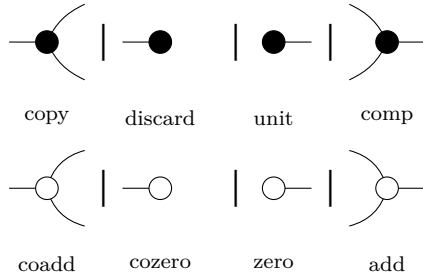
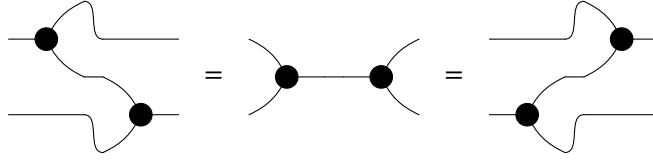


Figure 1: Frobenius additive (black) and co-additive (white) structure.

*subject to the Frobenius Law (both colors)*



## 2.2 Syntax: Symmetric Monoidal Theories

These constructions provide exactly the semantic foundation for the algebra we aim to build. In the same way that semantics is understood in linguistics and programming language theory [Win93], the semantics of a formal system assigns meaning to its expressions. More precisely, while syntax consists of the tokens and rules for forming and combining expressions, semantics is the interpretation assigned to each such expression — in our case, an optimization problem living in  $\mathbf{Rel}(Q)$ .

As outlined in the introduction, the central goal of this work is to show that optimization emerges naturally from algebraic and categorical structure. The key mechanism enabling this is the infimal composition present in the definition of the Cost category: composing two relations automatically performs variable elimination, which is precisely the operation underlying optimization.

To make this precise, we need both a semantics — provided by the categorical framework developed above — and a syntax: a rigorous, compositional language for expressing optimization problems symbolically. The map between those is a structure-preserving map, a functor, from the syntactic layer into the semantic one. The syntactic layer will be formalized as a Symmetric Monoidal Theory (SMT) [BSZ17], a presentation of a category by generators and equations, whose terms respect the compositional structure of a symmetric monoidal category.

**Definition 8** (Symmetric Monoidal Theory). *An SMT is a pair  $(\Sigma, E)$  such that  $\Sigma$  is a signature of operators  $o$  with arity  $m$  and coarity  $n$ .  $\Sigma$ -terms are defined inductively as the smallest collection of typed expressions  $t : m \rightarrow n$  closed under: generators  $o \in \Sigma$ ; identities  $\text{id}_n : n \rightarrow n$ ; composition  $a; b : m \rightarrow k$  for  $a : m \rightarrow n$ ,  $b : n \rightarrow k$ ; tensor product  $a \otimes b : m_1 + m_2 \rightarrow n_1 + n_2$  for  $a : m_1 \rightarrow n_1$ ,  $b : m_2 \rightarrow n_2$ ; and symmetries  $\sigma_{m,n} : m + n \rightarrow n + m$ . And  $E$  is a set of equations over the  $\Sigma$ -terms.*

Often, we will think of elements  $o : m \rightarrow n \in \Sigma$  as generators and represent them as string diagrams, the  $\Sigma$ -terms being the results of composing those generators horizontally and vertically inductively, and the set  $E$  as a collection of axiomatic equations over these string diagrams. The categorical counterpart of these terms is the free category generated by the SMT.

**Definition 9** (The Free category of an SMT). *The  $\text{Free}P_{(\Sigma, E)}$  is the category freely generated by  $(\Sigma, E)$  where the objects are natural numbers, morphisms are  $\Sigma$ -terms quotiented by  $E$ , with composition and tensor product inherited from the symmetric monoidal structure.  $\text{Free}P_{(\Sigma, E)}$  is in fact a  $\text{PROP}$ <sup>1</sup>.*

<sup>1</sup>For the definition of a PROP see [Lan98].



The  $\text{FreeP}_{(\Sigma, E)}$ , also denoted  $P\Sigma_{/E}$ ,<sup>2</sup> will be our **syntax** responsible for expressing our underlying semantics. For it to be considered a “useful” syntax we expect that it satisfies some properties, particularly we want it to be complete, sound and expressive. Without those, our algebra would be empty, since we would have no assurance that it actually says anything meaningful about what we desire to say.

**Definition 10.** *We will say that an SMT axiomatizes or represents a specific category, usually called the semantic category, when the following diagram commutes:*

$$\begin{array}{ccc}
 P\Sigma_{/E} = \text{FreeP}_{(\Sigma, E)} & \xrightarrow{\cong} & \text{Im}(\llbracket \cdot \rrbracket) \\
 \uparrow q & \nearrow s & \downarrow i \\
 \text{Syn} = P\Sigma & \xrightarrow{\llbracket \cdot \rrbracket} & \text{Sem}
 \end{array}$$

where  $P\Sigma$  is the category where objects are natural numbers and morphisms are  $\Sigma$ -terms from  $n \rightarrow m$ ;  $P\Sigma_{/E} = \text{FreeP}_{(\Sigma, E)}$  is the  $P\Sigma$  category with the morphisms quotiented by  $E$  and there is then a PROP morphism  $q : P\Sigma \rightarrow P\Sigma_{/E}$  witnessing this quotient; **Sem** is our semantic category (the Cost/Reward relations for example); and  $s$  is a full and faithful PROP morphism. Another way to say the same is to demand two things out of the SMT:

- *Soundness:*  $P\Sigma_{/E}$  is sound if  $c \stackrel{E}{=} d^3$  implies  $[c] = [d]$ .
- *Completeness:*  $P\Sigma_{/E}$  is complete if  $[c] = [d]$  implies  $c \stackrel{E}{=} d$ .
- *Expressivity:*  $P\Sigma_{/E}$  is expressive if for every morphism  $c$  in **Sem** there exists  $d \in P\Sigma_{/E}$  such that  $[d] = c$ .

Those three properties capture the idea that everything we can prove in the syntax holds in the semantics (soundness), everything that holds in the semantics is provable in the syntax (completeness), and that every morphism in the semantics has a syntactic representative (expressivity). Moreover, these properties denote equivalent characteristics about the interpretation map, both categorically and functionally, as depicted in Table 1.

### 2.3 Previous algebras

We now introduce the foundational theories for Graphical Algebra that serve as a foundation for the following sections, namely Graphical Linear Algebra [PRS22;

<sup>2</sup>During the work we will use  $\text{FreeP}_S$  and  $S$ , for whatever syntax  $S$  may be, interchangeably.

<sup>3</sup> $\stackrel{E}{=}$  means that this equality is provable by the equations in  $E$ .

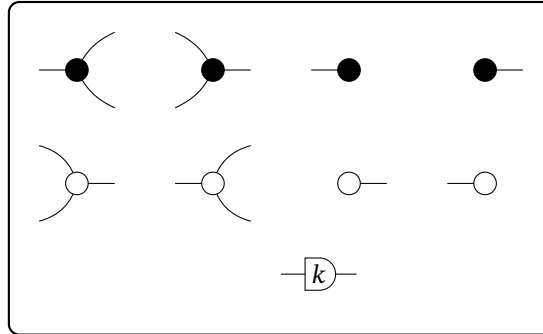
Table 1: Equivalence of properties for the interpretation map  $\llbracket - \rrbracket : P\Sigma_{/E} \rightarrow \mathbf{Sem}$

| Set theory    | Category theory  | Logic (SMT)                                                                                                      |
|---------------|------------------|------------------------------------------------------------------------------------------------------------------|
| Total         | Functor          | Soundness: $c \stackrel{E}{=} d \Rightarrow \llbracket c \rrbracket = \llbracket d \rrbracket$                   |
| Deterministic | Well-defined map | (Functionality of the interpretation)                                                                            |
| Injective     | Faithful         | Completeness: $\llbracket c \rrbracket = \llbracket d \rrbracket \Rightarrow c \stackrel{E}{=} d$                |
| Surjective    | Full             | Expressivity: $\forall f \in \mathbf{Sem}, \exists d \in P\Sigma_{/E} \text{ s.t. } \llbracket d \rrbracket = f$ |

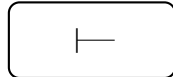
BSZ17], Graphical Affine Algebra [Bon+19], and Graphical Polyhedral Algebra [BDS21]. Each is an extension of the previous one, meaning there is a structure-preserving inclusion functor  $\mathbf{GLA} \hookrightarrow \mathbf{GAA} \hookrightarrow \mathbf{GPA}$ .

**Definition 11.** We will call **GLA** the presentation of the Graphical Linear Algebra SMT, in the same manner, **GAA** the SMT of Graphical Affine Algebra and **GPA** the SMT of Graphical Polyhedral Algebra. Below we show, separately, the set of generators (the signature  $\Sigma$ ) of each one of them.

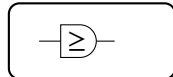
**GLA**



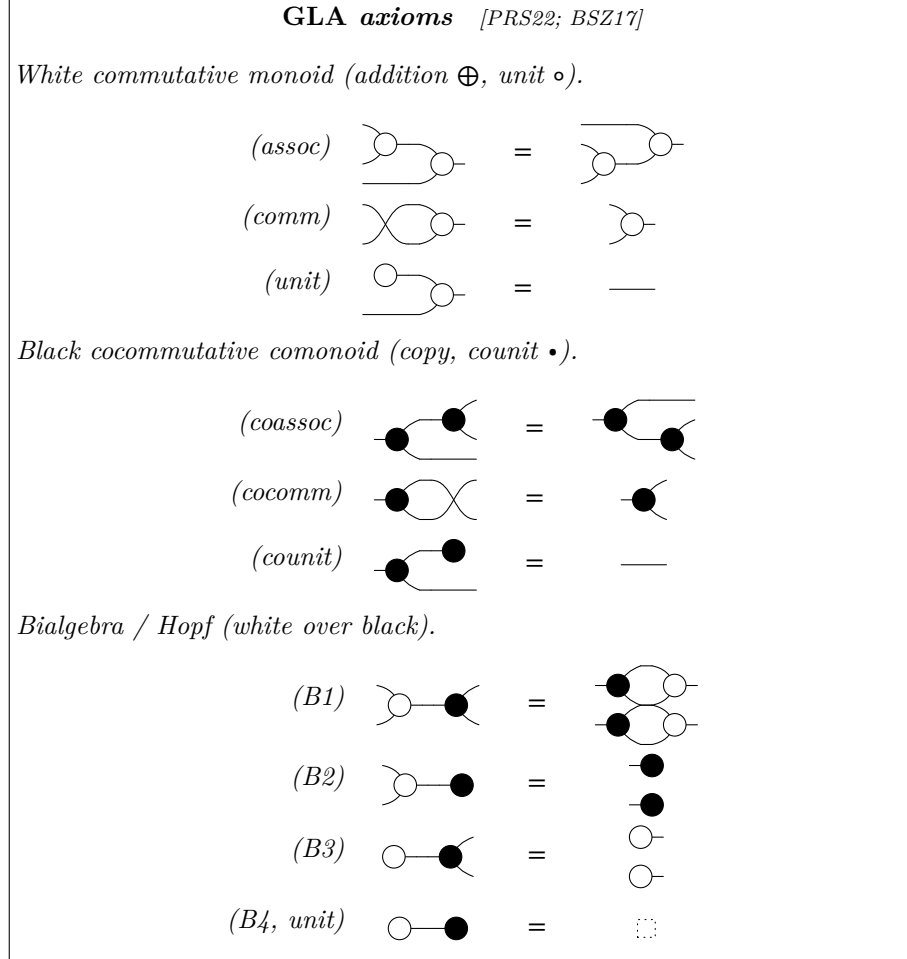
**GAA = GLA +**



**GPA = GAA +**



and the axioms, previously called the set  $E$  of equations for the SMT.



*Frobenius and special law (copy structure; the addition structure is dual).*

$$\begin{aligned}
 (\text{Frob}) \quad & \begin{array}{c} \text{---} \text{---} \text{---} \\ \diagup \quad \diagdown \\ \bullet \quad \bullet \\ \diagdown \quad \diagup \\ \text{---} \text{---} \end{array} = \begin{array}{c} \diagdown \quad \diagup \\ \bullet \quad \bullet \\ \diagup \quad \diagdown \end{array} \\
 (\text{special}) \quad & \begin{array}{c} \bullet \quad \bullet \\ \diagdown \quad \diagup \\ \text{---} \end{array} = \text{---}
 \end{aligned}$$

*Scalars and antipode (-1).*

$$\begin{aligned}
 (\text{s.add}) \quad & \begin{array}{c} \text{---} \bullet \\ \diagup \quad \diagdown \\ \boxed{k} \quad \boxed{l} \\ \diagdown \quad \diagup \\ \bullet \text{---} \end{array} = \boxed{k+l} \\
 (\text{s.mult}) \quad & \boxed{k} \text{---} \boxed{l} = \boxed{kl} \\
 (\text{antipode}) \quad & \begin{array}{c} \bullet \text{---} \boxed{\phantom{x}} \text{---} \bullet \\ \diagdown \quad \diagup \end{array} = \bullet \quad \circ
 \end{aligned}$$

### GAA axioms (in addition to GLA) [Bon+19]

The affine generator  $\vdash$  (a point  $0 \rightarrow 1$ , drawn as the empty box) is a comonoid homomorphism:

$$\begin{aligned}
 (\text{aff. copy}) \quad & \vdash \bullet = \begin{array}{c} \vdash \\ \vdash \end{array} \\
 (\text{aff. del}) \quad & \vdash \bullet = \begin{array}{c} \vdash \\ \vdash \end{array}
 \end{aligned}$$

### GPA axioms (in addition to GAA) [BDS21]

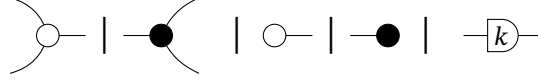
The order primitive  $\geq$  is a preorder compatible with the linear structure (transitivity; monoid and comonoid morphism;  $0 \geq 0$ ):

$$\begin{aligned}
 (\text{trans}) \quad & \boxed{\geq} \text{---} \boxed{\geq} = \boxed{\geq} \\
 (\text{mono. } \oplus) \quad & \begin{array}{c} \diagdown \quad \diagup \\ \bullet \quad \bullet \\ \diagup \quad \diagdown \end{array} \boxed{\geq} = \begin{array}{c} \boxed{\geq} \\ \boxed{\geq} \end{array} \begin{array}{c} \diagdown \quad \diagup \\ \bullet \quad \bullet \\ \diagup \quad \diagdown \end{array} \\
 (\text{mono. copy}) \quad & \bullet \begin{array}{c} \boxed{\geq} \\ \boxed{\geq} \end{array} \supseteq \boxed{\geq} \bullet \\
 (\text{zero}) \quad & \circ \boxed{\geq} = \circ
 \end{aligned}$$

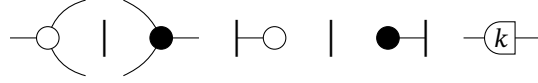
Together with reflexivity, antisymmetry and positivity of scalars; see [BDS21; BDZ21].

**Definition 12.** We also denote with  $\overrightarrow{(\cdot)}/\overleftarrow{(\cdot)}$  when we want to talk about the

algebra generated without the Frobenius structure. For example  $\overrightarrow{\mathbf{GLA}}$  is the algebra generated by

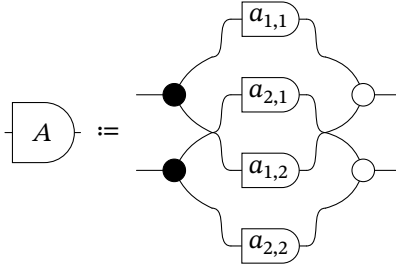


without their symmetric

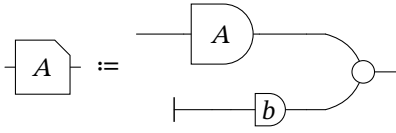


and the exactly opposite we denote as  $\overleftarrow{\mathbf{GLA}}$ .

Building on the above definition, we establish the following notation to ease the burden of future proofs. We will denote an abstract diagram from  $\overrightarrow{\mathbf{GLA}}$  as



Which denotes any matrix  $A$ , or, more formally, denotes the relation  $xAy \equiv Ax = y$ . Analogously, we also depict any diagram from  $\overleftarrow{\mathbf{GLA}}$  as



which denotes an affine transformation, also known as the relation  $xAy \equiv Ax + b = y$ . The equivalent hold for their dual form in  $\overleftarrow{\mathbf{GLA}}$  and  $\overleftarrow{\mathbf{GAA}}$ .

Our contribution is built on top of Graphical Polyhedral Algebra [BDS21; BDZ21]. Geometrically it goes beyond affine spaces, representing polyhedra as linear-inequality constraints. We begin by recalling the standard definitions.

**Definition 13.** A set  $P \subseteq \mathbb{R}^n$  is called a polyhedron if there exist  $A \in \mathbb{R}^{m \times n}$  and  $b \in \mathbb{R}^m$  such that  $P = \{x \in \mathbb{R}^n \mid Ax \geq b\}$ .

The corresponding semantic categories capture polyhedra as morphisms, with relational composition playing the role of variable elimination.

**Definition 14** (The prop **PolyRel** of polyhedra). *Let  $k$  be an ordered field. The prop  $\mathbf{PolyRel}_k$  has objects natural numbers  $n$ , morphisms  $n \rightarrow m$  the polyhedra  $R \subseteq k^n \times k^m$  of the form  $R = \{(x, y) \mid A(x, y)^\top + b \geq 0\}$ , composition relational composition, and monoidal product addition on objects, Cartesian product on morphisms.*

The central computational primitive of the polyhedral theory is Fourier–Motzkin Elimination (FME) [Sch86; Zie95], which projects a polyhedron onto a lower-dimensional space by eliminating one variable at a time. In **GPA** this is realized diagrammatically, after first establishing that the  $\geq$  generator is transitive, reflexive and antisymmetric.

**Proposition 2** (Fourier–Motzkin Elimination). *The following diagrammatic procedure is equivalent to performing the FME algorithm:*

$$\text{---} \boxed{\leq} \text{---} \boxed{A} \text{---} \boxed{B} \text{---} \boxed{\geq} \text{---} = \text{---} \boxed{A'} \text{---} \boxed{\geq} \text{---} \boxed{B'} \text{---}$$

Using FME, **GPA** admits a normal form, and applying the polar operator (the polyhedral analogue of Farkas duality, exchanging the  $H$ - and  $V$ -representations [BDZ21]) yields a second, conic-hull normal form. These drive the presentation theorems below, whose proofs we recall from [BDS21].

**Theorem 1.** **PolyRel** is presented by  $\text{FreeP}_{\mathbf{GPA}}$ .

This is the structural foundation on which our final contribution, **GLP**, is built: there, these constraints are combined with a linear objective to form full parametric linear programs.

### 3 Semantics of Linear Programs

Next, we introduce the categorical semantics of parametric linear programs (PLP). This section is self-contained. All results from the ordinary theory of linear programs, along with any categorical notions that are used here or later, are defined and proved within it.

Some results presented here, such as the existence of a standard form for PLP and certain equivalences between their epigraph and value function, will be of fundamental importance later when demonstrating the completeness of our algebra. Although those are known results from the literature, for the sake of comprehensibility we state and prove them over here.

Throughout this section, in order to facilitate the exposition of some ideas, we will often denote by  $\llbracket = \rrbracket$  that, two objects are equal under the functor  $\llbracket - \rrbracket$ , i.e., a diagram  $c \in \text{Ob}(\text{FreeP}_{\Sigma, E})$  is, under the interpretation map  $c \llbracket = \rrbracket \llbracket c \rrbracket$ .

#### 3.1 Preliminary Results in Linear Programming

We begin with several classical results that establish the tight relationship between piecewise affine convex functions, polyhedral functions, and parametric

linear programs. These will serve as the semantic foundation for the graphical algebra introduced in the following section.

**Definition 15** (Piecewise Affine Convex Function). *A function  $f : \mathbb{R}^n \rightarrow \mathbb{R}$  is called piecewise affine convex if there exist finitely many affine functions  $a_i^\top x + b_i$ ,  $i = 1, \dots, k$ , such that*

$$f(x) = \max_{i=1, \dots, k} \{a_i^\top x + b_i\}.$$

**Definition 16** (Polyhedral Function). *A function  $f : \mathbb{R}^n \rightarrow \mathbb{R} \cup \{+\infty\}$  is polyhedral if its epigraph  $\text{epi}(f) := \{(x, t) \mid t \geq f(x)\}$  is a polyhedron.*

**Definition 17** (RHS Parametric Linear Program). *A right-hand side parametric linear program has value function  $v(b) := \inf_x \{c^\top x \mid Ax \geq b\}$ .*

The following two propositions establish that all three notions above are equivalent descriptions of the same class of functions. This equivalence is the semantic cornerstone of the section: it is what allows us to treat polyhedral functions and parametric linear programs as morphisms in the same category.

**Proposition 3.** *A function is piecewise affine convex if and only if it is a polyhedral function.*

*Proof.* ( $\Rightarrow$ ) Let  $f(x) = \max_{i=1, \dots, k} (a_i^\top x + b_i)$ . Then

$$\text{epi}(f) = \bigcap_{i=1}^k \{(x, t) \mid t \geq a_i^\top x + b_i\},$$

which is an intersection of finitely many halfspaces, hence a polyhedron.

( $\Leftarrow$ ) If  $f$  is polyhedral, its epigraph is a polyhedron, so it can be written as an intersection of finitely many halfspaces:  $\text{epi}(f) = \bigcap_{i=1}^k \{(x, t) \mid t \geq a_i^\top x + b_i\}$ . Since  $f(x) = \inf\{t \mid (x, t) \in \text{epi}(f)\}$ , we obtain  $f(x) = \max_{i=1, \dots, k} (a_i^\top x + b_i)$ , so  $f$  is piecewise affine and convex.  $\square$

**Proposition 4.** *A function  $v : \mathbb{R}^m \rightarrow \mathbb{R} \cup \{+\infty\}$  is piecewise affine convex if and only if it is the value function of a right-hand side parametric linear program.*

*Proof.* ( $\Rightarrow$ ) Let  $v(b) = \max_{i=1, \dots, k} (y^i)^\top b$  and define  $Y := \text{conv}\{y^1, \dots, y^k\}$ . Then  $v(b) = \max_{y \in Y} b^\top y$ . Since  $Y$  is a polyhedron admitting the representation  $Y = \{y \mid A^\top y = c, y \geq 0\}$ , we have  $v(b) = \max_{y \in Y} b^\top y = \sup\{b^\top y \mid A^\top y = c, y \geq 0\}$ . By LP strong duality [Sch86; BV04], this dual program equals the primal  $\inf\{c^\top x \mid Ax \geq b\}$  (strong duality holds since  $Y$  is nonempty by construction), giving  $v(b) = \inf\{c^\top x \mid Ax \geq b\}$ .

( $\Leftarrow$ ) Let  $v(b) = \inf\{c^\top x \mid Ax \geq b\}$ . Dualising,  $v(b) = \sup\{b^\top y \mid A^\top y = c, y \geq 0\}$ . Since the feasible set  $Y = \{y \mid A^\top y = c, y \geq 0\}$  is a polyhedron, the supremum is attained at its extreme points  $y^1, \dots, y^k$  [Zie95], yielding  $v(b) = \max_{i=1, \dots, k} (y^i)^\top b$ . Thus  $v$  is piecewise affine convex.  $\square$

The following proposition is the key technical lemma for the completeness proof of **GLP**. It states that two parametric linear programs with the same value function must share the same epigraph polyhedron, providing the bridge between semantic equality and polyhedral equality that the completeness argument requires.

**Proposition 5.** *Let*

$$v_i(y) := \inf\{\mathbf{1}^\top x \mid A_i x \geq B_i y\}, \quad i = 1, 2,$$

where  $A_i \in \mathbb{R}^{m_i \times n_i}$  and  $B_i \in \mathbb{R}^{m_i \times k}$ . If  $v_1(y) = v_2(y)$  for all  $y \in \mathbb{R}^k$ , then both programs admit a common representation through the same epigraph polyhedron:

$$v_i(y) = \inf\{t \mid (y, t) \in E\}, \quad E := \text{epi}(v_1) = \text{epi}(v_2).$$

*Proof.* For each  $i = 1, 2$ , the epigraph of  $v_i$  satisfies

$$(y, t) \in \text{epi}(v_i) \iff \exists x : A_i x \geq B_i y, \mathbf{1}^\top x \leq t,$$

so  $\text{epi}(v_i)$  is the projection onto  $(y, t)$  of the polyhedron  $R_i = \{(x, y, t) \mid A_i x \geq B_i y, \mathbf{1}^\top x \leq t\}$ . Projections of polyhedra are polyhedra, so each  $\text{epi}(v_i)$  is a polyhedron. Since  $v_1 = v_2$  pointwise, we have  $\text{epi}(v_1) = \text{epi}(v_2) =: E$ . Setting  $E = \{(y, t) \mid Cy + dt \geq 0\}$ , we obtain  $v_i(y) = \inf\{t \mid Cy + dt \geq 0\}$  for both  $i = 1, 2$ .  $\square$

### 3.2 The semantic category **LPRel** and **PolyFun**

With the semantic equivalences established, we now define the two equivalent semantic categories for this algebra. The first, **LPRel**, describes morphisms as parametric linear programs; the second, **PolyFun**, describes them as polyhedral functions. The equivalence between the two follows immediately from the propositions above. We refer only to **LPRel**, but all results hold equally for **PolyFun**.

The broadest semantic setting is the category of bifunctions, which is exactly the Cost/Reward instance of **Rel(Q)** for the Extended-Lawvere quantale. Stein and collaborators [SS24; SS21] construct this category as the home of compositional convex optimization, where infimal convolution, conjugation, and variable elimination arise as categorical constructions; we recall only the bare construction we need.

**Definition 18** (Bifunction Category). *The category **BiFunc** has sets as objects, and a morphism  $F : A \rightarrow B$  is a function  $F : A \times B \rightarrow \overline{\mathbb{R}}$ . Composition is infimal convolution*

$$(G \circ F)(a, c) := \inf_{b \in B} (F(a, b) + G(b, c)),$$

with identity  $1_A(a, a') = 0$  if  $a = a'$  and  $+\infty$  otherwise. The tensor product is the Cartesian product on objects and addition on morphisms, with the singleton as unit.



**Remark 2.** The category **BiFunc** is exactly **Rel(Q)** for the Extended-Lawvere quantale  $(\mathbb{R}, \geq, +, 0)$ : a  $Q$ -valued relation  $R : A \times B \rightarrow \mathbb{R}$  is precisely a bifunction, and the composition law  $\bigvee_b R(a, b) \otimes S(b, c) = \inf_b (R(a, b) + S(b, c))$  is exactly infimal convolution. The Boolean-relations category embeds into **BiFunc** as a full subcategory via the indicator (characteristic) functor, so constraints and objectives are not fundamentally different objects: they are relations valued in different quantales, unified by the same compositional structure.

Composition is therefore precisely variable elimination: composing two relations marginalises over the intermediate variable by minimising the combined cost<sup>4</sup>

$$(S \circ R)(x, z) = \inf_y \{R(x, y) + S(y, z)\}.$$

Optimization is not imposed on the categorical structure from outside — it emerges from composition itself. Restricting to proper, convex, lower semicontinuous bifunctions yields the convex subcategory **CxBiFn**  $\subseteq$  **BiFunc** [Roc70], which is closed under infimal composition and is the natural home for the optimization problems we study. Linear programs are a finitely-presentable class of morphisms inside it.

**Definition 19** (The prop **LPRel** of Linear Programs). *The prop **LPRel** is defined as follows:*

- Objects are natural numbers  $n \in \mathbb{N}$ , interpreted as the space  $\mathbb{R}^n$ .
- A morphism  $n \rightarrow m$  is a right-hand side parametric linear program with value function

$$(x, y) \mapsto \inf_z \{c^\top z \mid Az \geq B[x \ y]^\top\},$$

where  $x \in \mathbb{R}^n$  and  $y \in \mathbb{R}^m$ .

- Composition is given by minimisation over the shared variable.
- The monoidal product on objects is addition  $n \otimes m := n + m$ , and on morphisms is direct sum.

**Definition 20** (The prop **PolyFun** of polyhedral functions). *The prop **PolyFun** is defined as follows:*

- Objects are natural numbers  $n \in \mathbb{N}$ , interpreted as the space  $\mathbb{R}^n$ .
- A morphism  $n \rightarrow m$  is a polyhedral function  $f : \mathbb{R}^{n+m} \rightarrow \overline{\mathbb{R}}$ .
- Composition is given by infimal convolution over the shared variable.
- The monoidal product on objects is addition  $n \otimes m := n + m$ , and on morphisms is  $f \otimes g := f + g$ .

---

<sup>4</sup>this notion is related to the idea of conditioning and marginalization in PPL [Ste+24; Mee+21]

**Remark 3.** *The two categories are isomorphic as PROPs:  $\mathbf{PolyFun} \cong \mathbf{LPRel}$ . This is a direct consequence of the equivalence between polyhedral functions and right-hand side parametric linear programs established in the propositions above. We will therefore use the two names interchangeably, choosing whichever description is more natural in a given context.*

## 4 Graphical Linear Programming

With the semantics in place, we define the Symmetric Monoidal Theory (SMT) that expresses it. In this section we present the base generators and the new equational setting of the algebra. Further results, such as syntactic derivations, normal form, and what we call "diagram solving", are demonstrated at the end of the section, providing some insightful applications of our theory.

The two graphical theories developed so far have each extended the categorical framework by adding a new class of morphisms. We now introduce the final and most expressive theory: a graphical algebra for parametric linear programs. Linear programs combine both the inequality structure of polyhedra and the notion of an objective function, and their value functions are precisely the polyhedral convex functions. The central result of this section is the presentation of a complete monoidal theory for right-hand side parametric linear programming.

The algebra **GLP** extends **GPA** with a single new generator whose semantics encodes the linear objective function  $c^\top x$ . As we will see more clearly later, this new generator plays a similar role to that of a "measurement" such that is simply exposing what is in the wire without modifying it. It lifts **GPA** from polyhedral constraints to full linear programs by attaching a linear cost to the feasible region.

5

**Definition 21 (GLP).** *We call **GLP** the symmetric monoidal theory obtained by extending **GPA** with the following generator:*

$$\llbracket \square \text{---} \rrbracket = (\bullet, x) \mapsto x$$

*together with the following axioms governing its interaction with the existing generators:*

$$\begin{array}{lll}
\text{(linearity)} & \begin{array}{c} \square \text{---} \\ \square \text{---} \end{array} & = \begin{array}{c} \square \text{---} \bigcirc \end{array} \\
\text{(zeroeq)} & \square \text{---} \bigcirc & = \text{---} \square \\
\text{(idemp)} & \square \text{---} \boxed{\geq} \text{---} & = \square \text{---}
\end{array}$$

---

<sup>5</sup>Analogous notion was developed by [Ste+24] with their gray arrow generator, introducing the  $\ell_2$  norm

The axioms represent expected properties of the linear cost function. The *linearity* axiom expresses  $f(x)+f(y) = f(x+y)$ , where  $f$  denotes the interpretation of  $\square\text{---}$  and the sum denotes vertical composition; composing two copies of  $\square\text{---}$  vertically is the same as applying a single  $\square\text{---}$  to the combined wires. The *idemp* axiom expresses the fundamental property of minimization: the infimum of a quantity over all values satisfying a lower bound is that lower bound itself,

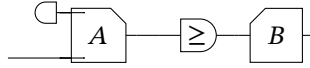
$$\inf_x \{x \mid x \geq y\} = y.$$

The name refers to the idempotency of the map  $\sigma \mapsto \sigma; \text{---}\overline{\geq}\text{---}$  on  $\Sigma$ -terms: applying  $\text{---}\overline{\geq}\text{---}$  after  $\square\text{---}$  has no effect, so  $\square\text{---}$  is a fixed point of this map. Finally, the *zeroeq* axiom captures the tautological case  $\inf\{x \mid x = 0\} = 0$ : the minimum of a linear cost subject to the equality  $x = 0$  is zero.

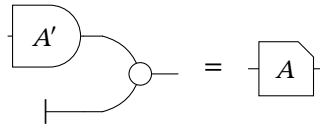
**Definition 22.** We call  $\text{FreeP}_{\mathbf{GLP}}$  the PROP freely generated by the signature and equations of **GLP**.

The normal form theorem for **GLP** extends the one for **GPA**: every diagram can be reduced to a **GPA** block encoding the constraints, composed with the new generator encoding the objective. The proof proceeds by structural induction, using the existing normal form for **GPA** as the inductive base.

**Theorem 2 (Normal Form).** For every diagram  $c \in \mathbf{GLP}$ , there exists a diagram  $c'$  in normal form such that  $c = c'$ :

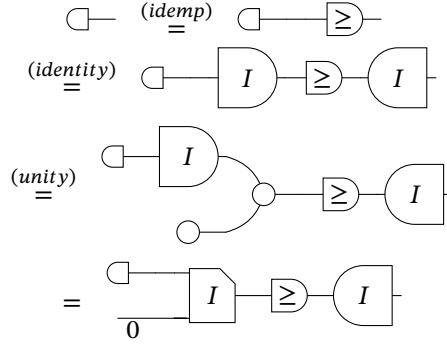


With  $A$  and  $b$  being an affine transformation, namely:

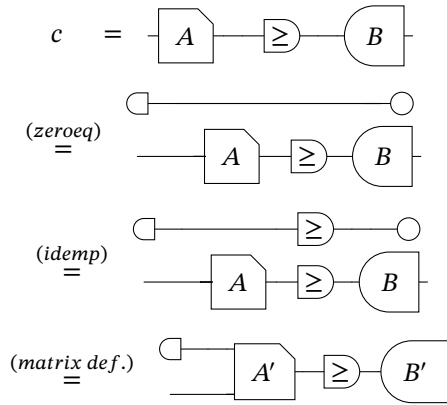


*Proof.* The proof is by structural induction. First we show the two base cases:

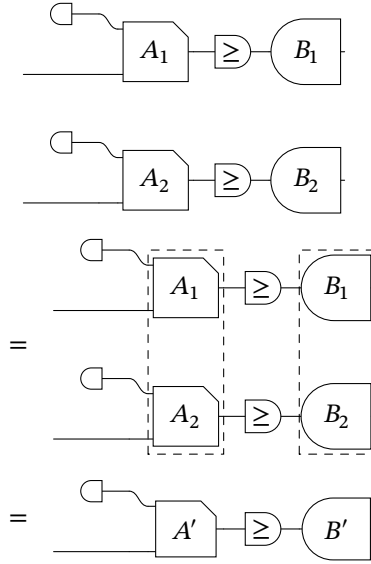
- The generator  $\square\text{---}$



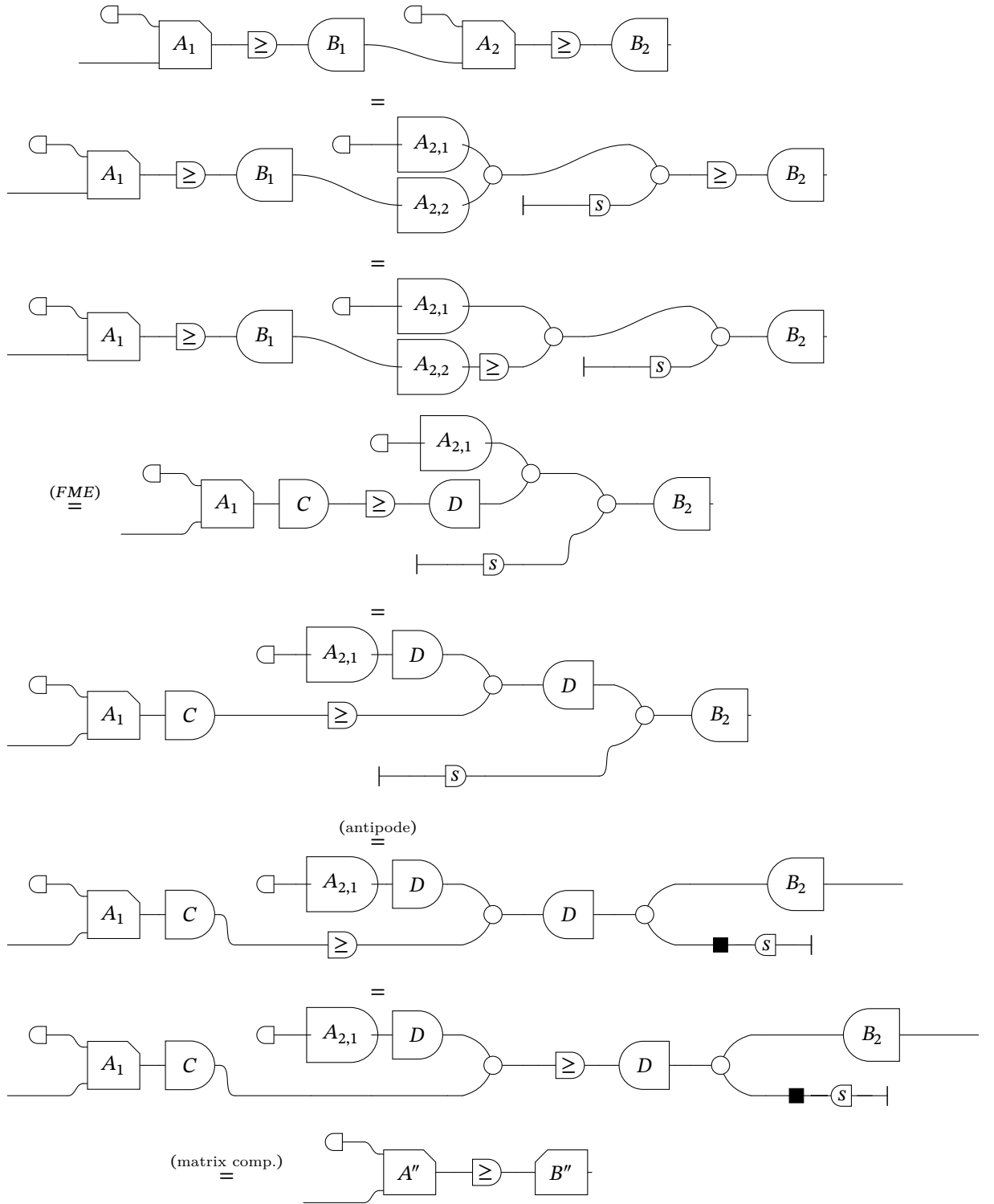
- A diagram  $c \in \mathbf{GPA}$  is a polyhedral, which by normal form [BDS21] is just  $\{Ax + a \geq By\}$ , thus:



For the inductive step, we verify that for any two diagrams  $c, d \in \mathbf{GLP}$  satisfying the inductive hypothesis, both horizontal and vertical composition preserve the normal form. Closure under vertical composition follows by stacking the two normal forms and collapsing the two independent  $A_i, \begin{smallmatrix} \text{---} \\ \text{---} \end{smallmatrix}, B_i$  blocks into a single block-diagonal matrix pair, as shown below:



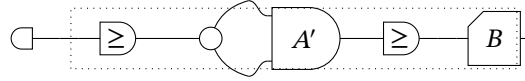
For horizontal composition, the two normal forms are joined along the wire connecting  $B_1$  to  $A_2$ ; applying Fourier-Motzkin Elimination to this pair recovers a fresh matrix-  $\begin{array}{c} \text{---} \end{array} \begin{array}{c} \geq \end{array} \text{---}$  -comatrix triple, after which the two resulting runs of matrices, one in each direction, compose into a single matrix and a single comatrix, recovering the normal form:



□

The following corollary reveals the geometric meaning of the normal form: the **GPA** part of every **GLP** diagram encodes the epigraph of the corresponding value function. This is the key structural insight connecting the syntax to the semantic equivalence of Proposition 5.

**Corollary 1** (Epigraph Form). *Given a diagram  $d \in \mathbf{GLP}$  in normal form, it can always be rewritten as a diagram  $d'$  of the form:*



where the semantics of the dotted diagram is

$$\{(t, y) \mid \exists x, A'x \geq By + c, t \geq x\}.$$

This is the epigraph of the original diagram  $d$

$$\begin{aligned} [d](y) &= \inf_x x \quad \text{s.t. } A'x \geq By + c \\ \text{epi}([d]) &= \{(t, y) \mid \exists x \leq t, A'x \geq By + c\} \end{aligned}$$

We call this the epigraph form of a linear program.

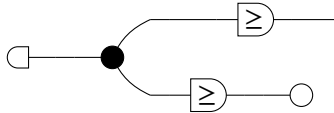
## 4.1 Examples

The expressivity of **GLP** becomes concrete when one considers specific classes of piecewise affine convex functions that arise naturally across mathematics and engineering. Each example below is a morphism in **LPRel** and can therefore be represented as a **GLP** diagram in normal form.

**Example 1** (ReLU activation). *The rectified linear unit  $f(y) = \max(0, y)$  is the elementary building block of modern neural networks. It is a piecewise affine convex function, and therefore a morphism in **GLP**, via the parametric linear program*

$$f(y) = \min_x \{x \mid x \geq 0, x \geq y\}.$$

As a **GLP** diagram, it consists of a single  $\square$  generator preceded by two inequality constraints: one bounding  $x$  from below by  $y$  and one enforcing non-negativity.



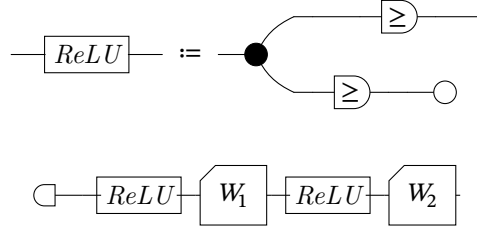
**Example 2** (Feedforward networks with ReLU activations). *A one-hidden-layer feedforward network with non-negative output weights produces a piecewise affine convex function:*

$$f(y) = \sum_{i=1}^k c_i \max(0, w_i^\top y + b_i), \quad c_i \geq 0.$$

*Each summand  $c_i \max(0, w_i^\top y + b_i)$  is piecewise affine convex (a scaled ReLU composed with an affine map), and their sum is again piecewise affine convex. The combined function is the value function of the parametric linear program*

$$f(y) = \min_{x_1, \dots, x_k} \sum_{i=1}^k c_i x_i \quad \text{s.t.} \quad x_i \geq 0, \quad x_i \geq w_i^\top y + b_i, \quad i = 1, \dots, k.$$

*The compositional structure of the network maps directly onto the monoidal structure of **GLP**: neurons within the same layer compose in parallel via the tensor product, while successive layers compose sequentially via categorical composition.*



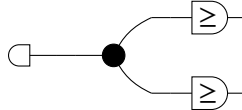
**Example 3** (Max-pooling). *Global max-pooling over a feature map  $(y_1, \dots, y_k)$ ,*

$$f(y_1, \dots, y_k) = \max_{i=1, \dots, k} y_i,$$

*is a piecewise affine convex function of the inputs. It is encoded as the linear program*

$$f(y) = \min_t \{t \mid t \geq y_i, \quad i = 1, \dots, k\},$$

*which is a **GLP** diagram of type  $k \rightarrow 0$  in which  $k$  inequality constraints feed into a single  $\square$  generator.*

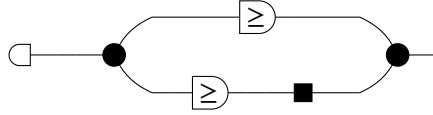




**Example 4** ( $\ell_1$  regularisation). The  $\ell_1$  norm  $f(y) = \|y\|_1 = \sum_{i=1}^n |y_i|$  is a canonical example in sparse optimisation [BV04]. It admits the LP representation

$$f(y) = \min_{t \in \mathbb{R}^n} \mathbf{1}^\top t \quad \text{s.t.} \quad t_i \geq y_i, \quad t_i \geq -y_i, \quad i = 1, \dots, n,$$

making  $\|y\|_1$  a **GLP** morphism of type  $n \rightarrow 0$ . The absolute value of each component,  $|y_i| = \max(y_i, -y_i)$ , is itself a piecewise affine convex function, and their sum is obtained via the additive structure inherited from **GLA**.



## 5 Syntax-Semantic Completeness

Finally, we prove that our semantics, namely that of Parametric Linear Programs, is fully axiomatized by the constructed SMT **GLP**. This section is divided into three parts:

- Proving Soundness: a check that every generator is correctly interpreted and all our axioms are sound with respect to the semantics.
- Expressivity that every semantic object, i.e. a *Parametric Linear Program*, has a standard normal form, and that this normal form is always the interpretation of some diagram.
- Completeness: through the epigraph form, any two identical PLPs can be reduced to the same polyhedron, which in turn makes their diagrams equal.

Together, these three properties establish that the semantics is fully axiomatised.

Throughout this section, in order to facilitate the exposition of some ideas, we will often denote by  $\llbracket - \rrbracket$  that two objects are equal under the functor  $\llbracket - \rrbracket$ , i.e., a diagram  $c \in \text{Ob}(\text{FreeP}_{\Sigma, E})$  is, under the interpretation map,  $c \llbracket = \rrbracket \llbracket c \rrbracket$ .

**Lemma 1** (Soundness). *If  $s = t$  in  $\text{FreeP}_{\text{GLP}}$ , then  $\llbracket s \rrbracket = \llbracket t \rrbracket$  in **PolyFun**.*

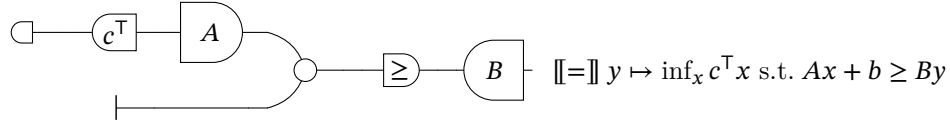
*Proof.* All axioms inherited from **GPA** are already sound by the results of the previous section. It suffices to verify that the new axioms involving the LP generator preserve the semantics, which follows by direct computation:

$$\begin{aligned}
(\text{linear.ge}) \quad & \square \text{---} \llbracket = \rrbracket (\bullet, y) \mapsto y = (\bullet, y) \mapsto \inf_{x \geq y} x \llbracket = \rrbracket \square \text{---} \boxed{\geq} \text{---} \\
(\text{lin.sum}) \quad & \begin{array}{c} \square \\ \square \end{array} \text{---} \llbracket = \rrbracket (\bullet, (y_1, y_2)) \mapsto y_1 + y_2 \llbracket = \rrbracket \square \text{---} \bigcirc \\
(\text{lin.zero}) \quad & \square \text{---} \bigcirc \llbracket = \rrbracket (\bullet, \bullet) \mapsto \inf_{x=0} x = 0 \llbracket = \rrbracket \bigcirc
\end{aligned}$$

□

**Lemma 2** (Expressivity). *For every  $f : m \rightarrow n$  in **LPRel** there exists a  $\Sigma$ -term  $s : m \rightarrow n$  in  $\text{Hom}_{\text{FreeP}_{\mathbf{GLP}}}(m, n)$  such that  $[s] = f$ .*

*Proof.* Since every RHS PLP admits a standard form, it suffices to show that every piecewise affine convex function is expressible in **GLP**. For an arbitrary such function  $f(y) = \inf_x c^\top x$ , where  $Ax + b \geq By$  the following diagram provides the required syntactic representative:



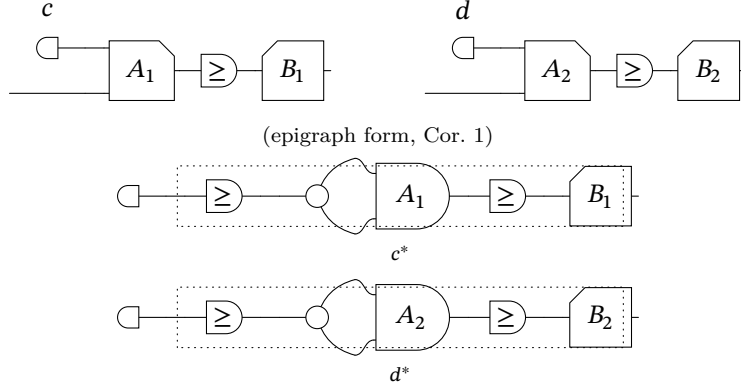
□

The final result is the completeness theorem for **GLP**. The proof brings together the main tools developed throughout: the epigraph form (Corollary 1), the shared epigraph property (Proposition 5), the completeness of **GPA**, and the FME-based normal form. It relies on the completeness of the **GPA** polyhedra case (Theorem 1); given that result, the argument below goes through.

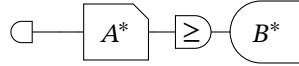
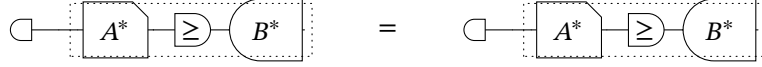
**Lemma 3** (Completeness). *If  $[c] = [d]$  in **PolyFun**, then  $c = d$  in  $\text{FreeP}_{\mathbf{GLP}}$ .*

*Proof.* Let  $v_1 = [c]$  and  $v_2 = [d]$ . By assumption  $v_1 = v_2$ , so Proposition 5 gives  $\text{epi}(v_1) = \text{epi}(v_2) =: E$ .

Diagrammatically,  $c$  and  $d$  start out as two distinct diagrams in normal form. By Corollary 1, each admits an epigraph form  $c = \square \text{---}; c^*$  and  $d = \square \text{---}; d^*$ , leaving the source generator  $\square \text{---}$  untouched and isolating everything else inside a dotted box: a pure **GPA** diagram  $c^*$  (resp.  $d^*$ ) whose denotation is the epigraph,  $[c^*] = [d^*] = E$  in **PolyRel**. Since  $c^*$  and  $d^*$  are **GPA** diagrams with the same denotation, completeness of **GPA** gives  $c^* = d^*$  in  $\text{FreeP}_{\mathbf{GPA}}$ ; in particular both reduce to the same **GPA** normal form. Composing this common diagram with the leftover  $\square \text{---}$  collapses  $c$  and  $d$  to the same **GLP** diagram:



(the dotted boxes  $c^*, d^*$  are **GPA** diagrams with  $[c^*] = [d^*] = E$ ; by **GPA** completeness both reduce to the same normal form)



Therefore  $c = \square$ ;  $c^* = \square$ ;  $d^* = d$ . □

As a consequence of the three lemmas above:

**Theorem 3.** *LPRel and PolyFun are both presented by  $\text{FreeP}_{\text{GLP}}$ .*

*Proof.* By the preceding three lemmas. □

## 6 Discussion and Future Work

We have built, through the addition of one generator and three axioms, an elegant and complete diagrammatic theory for Parametric Linear Programming. We argue that such a result not only unlocks a new way to interact with LP problems in an algebraic and computational way, but also provides a new perspective on optimization through the lens of a compositional framework. This lifts optimization, and LP problems in particular, from a functional setting to a relational-theoretic framework within the Extended-Lawvere Quantale category.

The development followed a strict linear discipline, adding exactly one new generator at a time and verifying at each step that the resulting algebra is sound, expressive, and complete with respect to a precisely defined semantic category. The fact that such a linear development is possible — that each new class of optimization problems requires only a single additional generator and a finite

set of axioms — is evidence that the compositional structure is not imposed artificially but present in the underlying mathematics. Polyhedral relations and quadratic relations are not merely two classes of optimization problems that happen to admit graphical representations; they are parallel enrichments of the same underlying affine structure, each adding a different kind of expressive power, and their common base is a theorem rather than an accident of notation.

The **GLP** theory has a natural extension to an even more expressive theory when enriched with more powerful generators. We believe that the class of quadratic programs could be, just as was done in this work, fully axiomatized by such a broader SMT. Another direction is the exploration of the functoriality of the bifunction adjoint (the Legendre–Fenchel transform extended to morphisms) [Wil15; Tou05]: this suggests a duality theory analogous to LP duality should be expressible diagrammatically as well.

A more applied direction concerns the computational implications of diagrammatic normal forms. If two optimization problems are equal as morphisms in **LPrel** if and only if their diagrams are related by the rewriting rules of **GLP**, then diagram simplification is problem simplification: a rewriting system on string diagrams is simultaneously a symbolic preprocessor for optimization solvers. Such an application of graphical algebras to theorem proving was already pointed to in other works [BDZ21; BDS21]. This work is a step toward fully expressing optimization within its own algebraic setting.

## References

- [BDS21] Filippo Bonchi, Alessandro Di Giorgio, and Paweł Sobociński. “Diagrammatic Polyhedral Algebra”. In: *41st IARCS Annual Conference on Foundations of Software Technology and Theoretical Computer Science (FSTTCS 2021)*. Vol. 213. Leibniz International Proceedings in Informatics (LIPIcs). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021, 40:1–40:18. DOI: 10.4230/LIPIcs.FSTTCS.2021.40. arXiv: 2105.10946.
- [BDZ21] Filippo Bonchi, Alessandro Di Giorgio, and Fabio Zanasi. “From Farkas’ Lemma to Linear Programming: an Exercise in Diagrammatic Algebra”. In: *9th Conference on Algebra and Coalgebra in Computer Science (CALCO 2021)*. Ed. by Fabio Gadducci and Alexandra Silva. Vol. 211. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021, 9:1–9:19. ISBN: 978-3-95977-212-9. DOI: 10.4230/LIPIcs.CALCO.2021.9. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.CALCO.2021.9>.
- [BF18] John C. Baez and Brendan Fong. “A Compositional Framework for Passive Linear Networks”. In: *Theory and Applications of Categories* 33.38 (2018), pp. 1158–1222. arXiv: 1504.05625. URL: <http://www.tac.mta.ca/tac/volumes/33/38/33-38.pdf>.

- [Bon+19] Filippo Bonchi et al. “Graphical Affine Algebra”. In: *Proceedings of the 34th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2019)*. Vancouver, BC, Canada: IEEE, 2019, pp. 1–12. DOI: 10.1109/LICS.2019.8785877.
- [BSZ17] Filippo Bonchi, Paweł Sobociński, and Fabio Zanasi. “Interacting Hopf Algebras”. In: *Journal of Pure and Applied Algebra* 221.1 (2017). Original complete axiomatisation of Graphical Linear Algebra, pp. 144–184. ISSN: 0022-4049. DOI: 10.1016/j.jpaa.2016.06.002. arXiv: 1403.7048.
- [BV04] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge, UK: Cambridge University Press, 2004. ISBN: 978-0-521-83378-3. URL: <https://web.stanford.edu/~boyd/cvxbook/>.
- [CD11] Bob Coecke and Ross Duncan. “Interacting Quantum Observables: Categorical Algebra and Diagrammatics”. In: *New Journal of Physics* 13 (2011), p. 043016. DOI: 10.1088/1367-2630/13/4/043016. arXiv: 0906.4725.
- [Coe+18] Bob Coecke et al. “Generalized Relations in Linguistics & Cognition”. In: *Theoretical Computer Science* (2018). Earlier conference version in WoLLIC 2017, LNCS vol. 10388, DOI: 10.1007/978-3-662-55386-2\_18. ISSN: 0304-3975. DOI: 10.1016/j.tcs.2018.03.008. URL: <https://www.sciencedirect.com/science/article/pii/S0304397518301476>.
- [Fre+25] Iago Leal de Freitas et al. *Generalizing the Invertible Matrix Theorem with Linear Relations using Graphical Linear Algebra*. 2025. arXiv: 2502.16783 [cs.SC]. URL: <https://arxiv.org/abs/2502.16783>.
- [FS19] Brendan Fong and David I. Spivak. “Hypergraph Categories”. In: *Journal of Pure and Applied Algebra* 223.11 (2019), pp. 4746–4777. ISSN: 0022-4049. DOI: 10.1016/j.jpaa.2019.02.014. arXiv: 1806.08304.
- [Han+24] Tyler Hanks et al. “A Compositional Framework for First-Order Optimization”. In: *arXiv preprint* (2024). arXiv: 2403.05711. URL: <https://arxiv.org/abs/2403.05711>.
- [JS91] André Joyal and Ross Street. “The Geometry of Tensor Calculus, I”. In: *Advances in Mathematics* 88.1 (1991), pp. 55–112. ISSN: 0001-8708. DOI: 10.1016/0001-8708(91)90003-P. URL: <https://www.sciencedirect.com/science/article/pii/000187089190003P>.
- [Lan98] Saunders Mac Lane. *Categories for the Working Mathematician*. 2nd. Vol. 5. Graduate Texts in Mathematics. Springer, New York, 1998. ISBN: 978-0-387-98403-2. DOI: 10.1007/978-1-4757-4721-8.

- [Law73] F. William Lawvere. “Metric Spaces, Generalized Logic, and Closed Categories”. In: *Rendiconti del Seminario Matematico e Fisico di Milano* 43.1 (1973). Reprinted in: *Reprints in Theory and Applications of Categories*, No. 1 (2002), pp. 1–37, pp. 135–166. DOI: 10.1007/BF02924844.
- [Mee+21] Jan-Willem van de Meent et al. *An Introduction to Probabilistic Programming*. 2021. arXiv: 1809.10756 [stat.ML]. URL: <https://arxiv.org/abs/1809.10756>.
- [PRS22] João Paixão, Lucas Rufino, and Paweł Sobociński. “High-level axioms for graphical linear algebra”. In: *Science of Computer Programming* 218 (2022), p. 102791. ISSN: 0167-6423. DOI: 10.1016/j.scico.2022.102791. URL: <https://www.sciencedirect.com/science/article/pii/S0167642322000247>.
- [PZ23] Robin Piedeleu and Fabio Zanasi. *An Introduction to String Diagrams for Computer Scientists*. 2023. arXiv: 2305.08768 [cs.LO]. URL: <https://arxiv.org/abs/2305.08768>.
- [Roc70] R. Tyrrell Rockafellar. *Convex Analysis*. Vol. 28. Princeton Mathematical Series. Princeton, New Jersey: Princeton University Press, 1970. ISBN: 978-0-691-01586-6. DOI: 10.1515/9781400873173.
- [Ros90] Kimmo I. Rosenthal. *Quantales and their Applications*. Vol. 234. Pitman Research Notes in Mathematics Series. Longman Scientific & Technical, 1990.
- [Sch86] Alexander Schrijver. *Theory of Linear and Integer Programming*. Wiley-Interscience Series in Discrete Mathematics and Optimization. John Wiley & Sons, 1986.
- [Sel10] Peter Selinger. “A Survey of Graphical Languages for Monoidal Categories”. In: *New Structures for Physics*. Ed. by Bob Coecke. Vol. 813. Lecture Notes in Physics. Springer, Berlin, Heidelberg, 2010, pp. 289–355. DOI: 10.1007/978-3-642-12821-9\_4. arXiv: 0908.3347.
- [SS21] Dario Stein and Sam Staton. “Compositional Semantics for Probabilistic Programs with Exact Conditioning”. In: *2021 36th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*. 2021, pp. 1–13. DOI: 10.1109/LICS52264.2021.9470552.
- [SS24] Dario Stein and Richard Samuelson. “Towards a Compositional Framework for Convex Analysis (with Applications to Probability Theory)”. In: *Foundations of Software Science and Computation Structures (FoSSaCS 2024)*. Vol. 14574. Lecture Notes in Computer Science. Springer, Cham, 2024, ? DOI: 10.1007/978-3-031-57228-9\_9. arXiv: 2312.02291.

- [Ste+24] Dario Stein et al. “Graphical Quadratic Algebra”. In: *Proceedings of the 39th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2024)*. Springer, Cham, 2024. DOI: 10.1007/978-3-032-11176-0\_18. arXiv: 2403.02284.
- [Tou05] Hugo Touchette. *Legendre-Fenchel Transforms in a Nutshell*. Tech. rep. Unpublished report. Started July 11, 2005; last revised October 16, 2014. School of Mathematical Sciences, Queen Mary, University of London, 2005. URL: <https://appliedmaths.sun.ac.za/~htouchette/archive/notes/lfth2.pdf>.
- [Wil15] Simon Willerton. *The Legendre-Fenchel Transform from a Category Theoretic Perspective*. 2015. arXiv: 1501.03791. URL: <https://arxiv.org/abs/1501.03791>.
- [Win93] Glynn Winskel. *The Formal Semantics of Programming Languages: An Introduction*. Foundations of Computing. Cambridge, Massachusetts: MIT Press, 1993. ISBN: 978-0-262-23169-5. URL: <https://mitpress.mit.edu/9780262731034/>.
- [Zie95] Günter M. Ziegler. *Lectures on Polytopes*. Vol. 152. Graduate Texts in Mathematics. Springer, 1995.